МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка

ЗВІТ

до лабораторної роботи №5
з дисципліни «Професійний Python»

Тема: «Обробка помилок, файли та серіалізація»

Виконав:
студент Левандівський Назар

Перевірив:
Куньо Іван Михайлович

Львів – 2026

Мета роботи

Навчитися створювати надійні програми, що передбачувано поводяться з помилками, файлами, конфігурацією та структурованими форматами: власна ієрархія винятків, exception chaining, context managers, logging, pathlib, CSV/JSON/YAML, streaming I/O, атомарний запис і валідація зовнішніх даних.

1. Постановка задачі

Варіант 1. Імпорт та експорт даних системи обліку студентів (продовження ЛР1–ЛР4). Поля: `student_id`, `first_name`, `last_name`, `group`, `average_grade`. Перевіряти ID, ПІБ і групу, `average_grade` у межах 0–100. Реалізувати CSV streaming import, strict/tolerant mode, JSON export, YAML configuration з мінімальним середнім балом, статистику valid/invalid і logging некоректних записів.

Таблиця 1 – Схема вхідних даних і правила перевірки

Поле	Тип	Правило	Код помилки
student_id	int	ціле число ≥ 1	bad_id, missing_field
first_name, last_name	str	непорожні; літери, дефіс, апостроф, пробіл	missing_field, bad_name
group	str	відповідає `group_pattern` із config (PI-21)	bad_group
average_grade	Decimal	число від 0 до 100, округлення до сотих	bad_number, grade_out_of_range

Приклад `data/students.csv`: 20 записів, із них 7 некоректних і 2 коректні, але нижчі за мінімальний бал 60 (перші 9 рядків):

```
student_id,first_name,last_name,group,average_grade
1,Анна,Коваль,PI-21,91.5
2,Олег,Мельник,PI-21,84.0
3,Ірина,Бондар,PI-22,95.2
4,,Шевченко,PI-21,88.7
5,Марія,Бойко,PI-22,105
abc,Тарас,Гриценко,PI-23,72.4
7,Софія,Гнатюк,PI-22,66.8
8,Дмитро,Ткаченко,PI-2,58.9
```

2. Структура проєкту та ієрархія винятків

```
Lab_05/  config.yaml, pyproject.toml, README.md
├── data/          students.csv | .jsonl | .yaml
├── output/  logs/ (створюються програмою)
├── src/reliable_importer/  exceptions, config, logging_config, readers,
│                           validators, exporters, services, utils, models, main
├── tests/         37 тестів (unittest)
└── experiments/   run_experiments.py -> results/
```

Усі очікувані помилки успадковують `ApplicationError`, тому `main()` відрізняє їх від непередбачених (код 1 проти 2). `RecordValidationError` несе номер рядка, поле й код, але не значення полів: повідомлення безпечно писати в log.

```
ApplicationError
├── ConfigurationError
└── DataError
    ├── DataImportError
    │   └── UnsupportedFormatError
    ├── RecordValidationError
    └── DataExportError
```

```
class RecordValidationError(DataError):
    def __init__(
        self,
        message: str,
        *,
        line_number: int | None = None,
        field: str | None = None,
        code: str = "invalid",
    ) -> None:
        super().__init__(message)
        self.message = message
        self.line_number = line_number
        self.field = field
        self.code = code

    def __str__(self) -> str:
        where = f"рядок {self.line_number}" if self.line_number is not None else "запис"
        column = f", поле {self.field!r}" if self.field else ""
        return f"{where}{column}: {self.message} [{self.code}]"
```

Exception chaining. Технічна помилка розбору числа перетворюється на доменну через `raise ... from error`, і оригінальна причина зберігається в `__cause__`. Для оцінки `wrong` причиною є `decimal.InvalidOperation`, для ідентифікатора `abc` – `ValueError` (обидва випадки перевіряють тести). Traceback експерименту 3 показує обидві ланки:

```
@staticmethod
def _grade(row: RawRow, line_number: int) -> Decimal:
    value = row.get("average_grade")
    if value is None or isinstance(value, bool):
        raise RecordValidationError(
            "поле відсутнє",
            line_number=line_number,
            field="average_grade",
            code="missing_field",
        )
    try:
        grade = Decimal(str(value).strip())
    except InvalidOperation as error:
        raise RecordValidationError(
            "значення не є числом",
            line_number=line_number,
            field="average_grade",
            code="bad_number",
        ) from error
    if not grade.is_finite():
        raise RecordValidationError(
            "значення не є скінченним числом",
            line_number=line_number,
            field="average_grade",
            code="bad_number",
        )
    if not 0 <= grade <= 100:
        raise RecordValidationError(
            "середній бал має бути від 0 до 100",
            line_number=line_number,
            field="average_grade",
            code="grade_out_of_range",
        )
    return grade.quantize(HUNDREDTH, ROUND_HALF_UP)
```

```
PS C:\Github\prof_python\Lab_05> python experiments/chaining_demo.py
Traceback (most recent call last):
  File "src\reliable_importer\validators.py", line 91, in _grade
    grade = Decimal(str(value).strip())
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^
decimal.InvalidOperation: [<class 'decimal.ConversionSyntax'>]

The above exception was the direct cause of the following exception:
```

```
Traceback (most recent call last):
  File "experiments\chaining_demo.py", line 22, in <module>
    RecordValidator(CONFIG).validate(2, ROW)
  File "src\reliable_importer\validators.py", line 29, in validate
    grade = self._grade(row, line_number)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "src\reliable_importer\validators.py", line 93, in _grade
    raise RecordValidationError(
reliable_importer.exceptions.RecordValidationError: рядок 2, поле 'average_grade': значення не є числом
[bad_number]
```

3. Конфігурація

```
schema_version: 1

input:
  path: data/students.csv
  format: auto

output:
  path: output/students.json
  format: json
  errors_path: output/errors.csv
  summary_path: output/summary.json
  backup: true

processing:
  skip_invalid: true
  minimum_grade: 60
  group_pattern: 'PI-\d{2}'
  progress_every: 10000

logging:
  level: INFO
  path: logs/application.log
  max_bytes: 1048576
  backup_count: 3
```

`load_config` читає YAML через `safe_load`, застосовує змінні середовища (`STUDENTIO_SKIP_INVALID`, `STUDENTIO_MINIMUM_GRADE` тощо), будує frozen dataclass і перетворює всі технічні помилки на `ConfigurationError` із chaining. `validate_config` (fail-fast) перевіряє версію схеми, `minimum_grade` у межах 0–100, формати, рівень логування, наявність вхідного файла й те, що вхід і вихід не збігаються. Параметр `minimum_grade` керує експортом: студенти з нижчим балом вважаються коректними, але до JSON не потрапляють.

```
def load_config(path: Path, environ: Mapping[str, str] | None = None) -> AppConfig:
    try:
        with path.open("r", encoding="utf-8") as file:
            raw = yaml.safe_load(file)
    except FileNotFoundError as error:
        raise ConfigurationError(f"Файл конфігурації не знайдено: {path}") from error
    except yaml.YAMLError as error:
        raise ConfigurationError(f"Некоректний YAML у {path.name}") from error
    except (OSError, UnicodeDecodeError) as error:
        raise ConfigurationError(f"Не вдалося прочитати {path}") from error

    if not isinstance(raw, dict):
        raise ConfigurationError("Корінь конфігурації має бути словником")
    apply_env_overrides(raw, os.environ if environ is None else environ)
    try:
        config = _build(raw, base=path.resolve().parent)
    except (KeyError, TypeError, ValueError, InvalidOperation, re.error) as error:
        raise ConfigurationError(f"Некоректна структура конфігурації: {error!r}") from error
    validate_config(config)
    return config
```

4. Logging

Лог пишеться у файл із ротацією (`RotatingFileHandler`) і в консоль. Повідомлення про некоректні записи містять лише номер рядка, поле,

причину й код помилки, але не значення полів: імена студентів у log не потрапляють (це перевірено на справжньому файлі log під час генерації звіту й тестом). Приклад log – у розділі 7.

```
def configure_logging(config: LoggingConfig) -> None:
    config.path.parent.mkdir(parents=True, exist_ok=True)
    file_handler = RotatingFileHandler(
        config.path,
        maxBytes=config.max_bytes,
        backupCount=config.backup_count,
        encoding="utf-8",
    )
    console_handler = logging.StreamHandler(sys.stderr)
    formatter = logging.Formatter(FORMAT)
    for handler in (file_handler, console_handler):
        handler.setFormatter(formatter)
    logging.basicConfig(
        level=getattr(logging, config.level),
        handlers=[file_handler, console_handler],
        force=True,
    )

@contextmanager
def logged_operation(operation: str) -> Iterator[None]:
    logger.info("%s: початок", operation)
    start = time.perf_counter()
    try:
        yield
    except Exception:
        logger.error("%s: перервано через %.3f с", operation, time.perf_counter() - start)
        raise
    else:
        logger.info("%s: завершено за %.3f с", operation, time.perf_counter() - start)
```

5. Streaming reader, validation, strict / tolerant

Єдиний інтерфейс `Importer` (Protocol) має реалізації для CSV, JSON Lines (потокові), JSON і YAML (читаються цілком); формат обирається за розширенням. CSV читається рядок за рядком, а технічні помилки файлової системи, кодування та синтаксису перетворюються на `DataImportError`:

```
def read(self, path: Path) -> RowStream:
    try:
        with path.open("r", encoding="utf-8-sig", newline="") as file:
            reader = csv.DictReader(file)
            missing = set(REQUIRED_COLUMNS) - set(reader.fieldnames or ())
            if missing:
                raise DataImportError(f"{path.name}: бракує колонок {sorted(missing)}")
            for row in reader:
                yield reader.line_num, dict(row)
    except FileNotFoundError as error:
        raise DataImportError(f"Вхідний файл не знайдено: {path.name}") from error
    except UnicodeDecodeError as error:
        raise DataImportError(f"{path.name}: файл не в кодуванні UTF-8") from error
    except csv.Error as error:
        raise DataImportError(f"{path.name}: пошкоджений CSV") from error
    except OSError as error:
        raise DataImportError(f"Не вдалося прочитати {path.name}") from error
```

Валідатор перевіряє поля у фіксованому порядку, правила беруться з конфігурації:

```
def validate(self, line_number: int, row: RawRow) -> StudentRecord:
    student_id = self._student_id(row, line_number)
    first_name = self._name(row, "first_name", line_number)
    last_name = self._name(row, "last_name", line_number)
    group = self._text(row, "group", line_number)
    if not self._config.group_pattern.fullmatch(group):
        raise RecordValidationError(
            "код групи не відповідає формату",
            line_number=line_number,
            field="group",
            code="bad_group",
        )
```

```
    grade = self._grade(row, line_number)
    return StudentRecord(line_number, student_id, first_name, last_name, group, grade)

@staticmethod
def _student_id(row: RawRow, line_number: int) -> int:
    value = row.get("student_id")
    if value is None or isinstance(value, bool):
        raise RecordValidationError(
            "поле відсутнє", line_number=line_number, field="student_id", code="missing_field"
        )
    try:
        student_id = int(str(value).strip())
    except ValueError as error:
        raise RecordValidationError(
            "ідентифікатор не є цілим числом",
            line_number=line_number,
            field="student_id",
            code="bad_id",
        ) from error
    if student_id < 1:
        raise RecordValidationError(
            "ідентифікатор має бути додатним",
            line_number=line_number,
            field="student_id",
            code="bad_id",
        )
    return student_id
```

Поведінку з некоректним записом визначає об'єкт-політика: strict піднімає виняток (fail-fast), tolerant пише warning у log і рядок в `errors.csv`. Генератори `import_records` і `filter_by_grade` ведуть статистику й віддають записи по одному:

```
class StrictPolicy:
    def on_invalid(self, error: RecordValidationError, row: RawRow) -> None:
        raise error

class TolerantPolicy:
    def __init__(self, report: ErrorReport) -> None:
        self._report = report

    def on_invalid(self, error: RecordValidationError, row: RawRow) -> None:
        logger.warning("Некоректний запис пропущено: %s", error)
        self._report.add(error, row.get("student_id"))

def import_records(
    rows: Iterable[tuple[int, RawRow]],
    validator: RecordValidator,
    policy: ValidationPolicy,
    stats: ImportStatistics,
    progress_every: int = 10_000,
) -> Generator[StudentRecord, None, None]:
    for line_number, row in rows:
        stats.total += 1
        try:
            record = validator.validate(line_number, row)
        except RecordValidationError as error:
            stats.invalid += 1
            stats.errors[error.code] += 1
            policy.on_invalid(error, row)
        else:
            stats.valid += 1
            yield record
        if stats.total % progress_every == 0:
            logger.info("Прогрес: %d записів (некоректних %d)", stats.total, stats.invalid)

def filter_by_grade(
    records: Iterable[StudentRecord],
    minimum_grade: Decimal,
    stats: ImportStatistics,
    groups: Counter[str],
) -> Generator[StudentRecord, None, None]:
    for record in records:
        if record.average_grade < minimum_grade:
            stats.below_minimum += 1
            continue
        groups[record.group] += 1
        yield record
```

6. JSON serialization, atomic export, context managers

`JsonExporter` пише JSON-масив потоково, а Decimal серіалізується числом (custom encoder): оцінка має не більше двох знаків, тому `float` її не спотворює. `AtomicWriter` – власний class-based context manager: дані пишуться у тимчасовий файл у тому ж каталозі; лише після успішного блоку with виконуються `fsync`, резервна копія `.bak` і атомарний `os.replace` (з `retry` на випадок тимчасового блокування файла у Windows); при винятку тимчасовий файл видаляється. Generator-based `logged_operation` журналює тривалість операції.

```
class DecimalJSONEncoder(json.JSONEncoder):
    def default(self, o: object) -> object:
        if isinstance(o, Decimal):
            return float(o)
        return super().default(o)

class JsonExporter:
    def write(self, records: Iterable[StudentRecord], file: TextIO) -> int:
        count = 0
        file.write("[")
        for record in records:
            file.write(",\n  " if count else "\n  ")
            file.write(json.dumps(record.to_dict(), cls=DecimalJSONEncoder, ensure_ascii=False))
            count += 1
        file.write("\n]\n" if count else "]\n")
        return count
```

```
def __exit__(
    self,
    exc_type: type[BaseException] | None,
    exc: BaseException | None,
    traceback: TracebackType | None,
) -> None:
    assert self._file is not None and self._temp is not None
    try:
        try:
            if exc_type is None:
                self._file.flush()
                os.fsync(self._file.fileno())
        finally:
            self._file.close()
        if exc_type is not None:
            logger.warning("%s: запис скасовано, тимчасовий файл видалено", self.path.name)
            return
        if self.backup and self.path.exists():
            shutil.copy2(self.path, self.path.with_name(self.path.name + ".bak"))
        replace_file(self._temp, self.path)
    except OSError as error:
        raise DataExportError(f"Неможливо записати {self.path.name}") from error
    finally:
        self._temp.unlink(missing_ok=True)
```

У `run_import_export` писачі й генератори реєструються в `ExitStack`, причому генератори – через `contextlib.closing`: при зупинці strict-режиму посередині файла вхідний CSV закривається одразу, а не після збирання сміття (у Windows відкритий файл не можна видалити; це перевіряє тест `test_strict_abort_releases_input_file`).

7. Виконання програми

```
PS C:\Github\prof_python\Lab_05> python -m reliable_importer.main
2026-11-09 13:36:19,108 | INFO    | reliable_importer.utils | Імпорт/експорт студентів: початок
2026-11-09 13:36:19,108 | INFO    | reliable_importer.services | Імпорт students.csv (режим tolerant)
2026-11-09 13:36:19,108 | INFO    | reliable_importer.exporters | Створюється каталог для students.json
2026-11-09 13:36:19,109 | WARNING | reliable_importer.services | Некоректний запис пропущено: рядок 5, поле
'first_name': поле відсутнє або порожнє [missing_field]
2026-11-09 13:36:19,109 | WARNING | reliable_importer.services | Некоректний запис пропущено: рядок 6, поле
'average_grade': середній бал має бути від 0 до 100 [grade_out_of_range]
```

```
2026-11-09 13:36:19,113 | WARNING | reliable_importer.services | Некоректний запис пропущено: рядок 7, поле
'student_id': ідентифікатор не є цілим числом [bad_id]
2026-11-09 13:36:19,113 | WARNING | reliable_importer.services | Некоректний запис пропущено: рядок 9, поле
'group': код групи не відповідає формату [bad_group]
2026-11-09 13:36:19,113 | WARNING | reliable_importer.services | Некоректний запис пропущено: рядок 12, поле
'average_grade': значення не є числом [bad_number]
2026-11-09 13:36:19,113 | WARNING | reliable_importer.services | Некоректний запис пропущено: рядок 15, поле
'first_name': ім'я містить недозволені символи [bad_name]
2026-11-09 13:36:19,113 | WARNING | reliable_importer.services | Некоректний запис пропущено: рядок 18, поле
'average_grade': середній бал має бути від 0 до 100 [grade_out_of_range]
2026-11-09 13:36:19,124 | INFO    | reliable_importer.utils | Імпорт/експорт студентів: завершено за 0.016 с
2026-11-09 13:36:19,124 | INFO    | reliable_importer | Підсумок: total=20, valid=13, invalid=7,
below_minimum=2, exported=11, помилки={'missing_field': 1, 'grade_out_of_range': 2, 'bad_id': 1, 'bad_group':
1, 'bad_number': 1, 'bad_name': 1}
```

Рисунок 1 – Tolerant mode: 20 записів, 11 експортовано, 7 некоректних, 2 нижче мінімуму

```
PS C:\Github\prof_python\Lab_05> $env:STUDENTIO_SKIP_INVALID = "false"; python -m reliable_importer.main
2026-11-09 13:36:19,252 | INFO    | reliable_importer.config | Параметр processing.skip_invalid перевизначено
змінною STUDENTIO_SKIP_INVALID
2026-11-09 13:36:19,253 | INFO    | reliable_importer.utils | Імпорт/експорт студентів: початок
2026-11-09 13:36:19,253 | INFO    | reliable_importer.services | Імпорт students.csv (режим strict)
2026-11-09 13:36:19,254 | WARNING | reliable_importer.exporters | errors.csv: запис скасовано, тимчасовий файл
видалено
2026-11-09 13:36:19,254 | WARNING | reliable_importer.exporters | students.json: запис скасовано, тимчасовий
файл видалено
2026-11-09 13:36:19,255 | ERROR   | reliable_importer.utils | Імпорт/експорт студентів: перервано через 0.002 с
2026-11-09 13:36:19,255 | ERROR   | reliable_importer | Помилка застосунку: рядок 5, поле 'first_name': поле
відсутнє або порожнє [missing_field]
2026-11-09 13:36:19,255 | ERROR   | reliable_importer | Оброблено до зупинки: {'total': 4, 'valid': 3,
'invalid': 1, 'below_minimum': 0, 'exported': 0, 'errors_by_code': {'missing_field': 1}, 'exported_by_group':
{}}; вихідні файли не змінено
```

Рисунок 2 – Strict mode через змінну середовища: зупинка на рядку 5, вихідні файли не змінено, exit code 1

Ті самі рядки записано у `logs/application.log`. Нижче – `output/errors.csv`, початок `output/students.json` і `summary.json` після tolerant-запуску:

```
line,student_id,field,code,message
5,4,first_name,missing_field,поле відсутнє або порожнє
6,5,average_grade,grade_out_of_range,середній бал має бути від 0 до 100
7,abc,student_id,bad_id,ідентифікатор не є цілим числом
9,8,group,bad_group,код групи не відповідає формату
12,11,average_grade,bad_number,значення не є числом
15,14,first_name,bad_name,ім'я містить недозволені символи
18,17,average_grade,grade_out_of_range,середній бал має бути від 0 до 100
```

```
[
  {"student_id": 1, "first_name": "Анна", "last_name": "Коваль", "group": "PI-21", "average_grade":
91.5},
  {"student_id": 2, "first_name": "Олег", "last_name": "Мельник", "group": "PI-21", "average_grade":
84.0},
  ...

{"input": "students.csv", "mode": "tolerant", "total": 20, "valid": 13, "invalid": 7, "below_minimum":
2, "exported": 11, "errors_by_code": {"missing_field": 1, "grade_out_of_range": 2, "bad_id": 1,
"bad_group": 1, "bad_number": 1, "bad_name": 1}, "exported_by_group": {"PI-21": 3, "PI-22": 4, "PI-
23": 2, "PI-24": 2}}
```

8. Експерименти

Таблиця 2 – Експеримент 1: 1000 валідних + 10 некоректних записів

Параметр	Strict	Tolerant
Оброблено записів	51	1010
Valid	50	1000
Invalid	1	10
Exported	0	1000

Параметр	Strict	Tolerant
Завершення application	exit 1: зупинка: рядок 52, поле 'group': код групи не відповідає формату [bad_group]; output не створено	exit 0: успішно

Strict доцільний там, де частковий імпорт неприпустимий (наприклад, нарахування за оцінками): краще не імпортувати нічого, ніж половину. Tolerant – для масових даних, де некоректні рядки виправляють окремо за `errors.csv`.

Таблиця 3 – Експеримент 2: streaming vs eager (час – найкращий із 3 почергових запусків, пам'ять – пік tracemalloc)

Records	Eager time, с	Streaming time, с	Streaming / eager	Eager peak RAM, МБ	Streaming peak RAM, МБ
10 000	0.070	0.066	0.94	8.8	0.10
100 000	0.733	0.649	0.89	87.0	0.10
500 000	3.842	3.245	0.84	435.5	0.10

Пам'ять eager росте лінійно (8.8 → 435.5 МБ), streaming залишається ≈0.10 МБ – у 4 183 рази менше на 500 000 записів. За часом на 10 000 записів streaming швидший (streaming/eager = 0.94), на 500 000 записів – швидший (streaming/eager = 0.84): парсинг, перевірка й серіалізація в обох варіантах однакові, тому виграш у часі не гарантований, а в пам'яті – гарантований. Заміри часу робилися почергово (A, B, A, B), щоб зміна частоти процесора впливала на обидва варіанти однаково.

Експеримент 4 (atomic output). Після успішного експорту запущено експорт із exporter, що падає з `OSError` на 6-му записі. Результат: `DataExportError: Помилка запису students.json <- OSError`; попередній файл не змінено (SHA-256 збігається); тимчасових файлів: 0; у log – «students.json: запис скасовано, тимчасовий файл видалено». `DataExportError` – нащадок `ApplicationError`, тому `main()` перехоплює його й повертає код 1.

Таблиця 4 – Експеримент 5: помилки конфігурації

Випадок	Exit	Повідомлення в log
відсутній config.yaml	1	Помилка застосунку: Файл конфігурації не знайдено: data\generated\exp5\absent.yaml (причина: FileNotFoundError)
неправильний YAML	1	Помилка застосунку: Некоректний YAML у broken.yaml (причина: ParserError)
немає ключа input	1	Помилка застосунку: Некоректна структура конфігурації: KeyError('input') (причина: KeyError)
minimum_grade = 150	1	Помилка застосунку: minimum_grade має бути числом від 0 до 100
input file відсутній	1	Помилка застосунку: Вхідний файл не знайдено: nope.csv
output directory не існує	0	Створюється каталог для students.json

9. Перевірка правильності

37 тестів перевіряють завантаження й семантичну перевірку конфігурації (13 некоректних випадків, змінні середовища, межі minimum_grade), кожен код помилки валідатора та граничні значення,

chaining, відсутність значень у повідомленнях, усі формати й зламані файли, strict/tolerant, фільтр за мінімальним балом, атомарність, backup, retry, відсутність імен у log і коди завершення `main`.

Перевірка	**Результат**
`python -m unittest discover -s tests`	Ran 37 tests in 1.340s — OK
`mypy (strict, src + tests)`	Success: no issues found in 14 source files
`ruff check src tests`	All checks passed!

Висновки

Створено надійний модуль імпорту/експорту студентів: дані перевіряються на межі системи, кожна очікувана помилка має свій тип і код, причина зберігається через chaining, ресурси звільняють context managers, а атомарний запис, як показав експеримент, не пошкоджує попередній output. Поведінка (strict/tolerant, мінімальний бал, формати) змінюється через YAML або змінні середовища без редагування коду, log не містить персональних даних, а streaming тримає пам'ять сталою незалежно від розміру файла.